CHAPTER 1

STRUCTURES AND POINTERS

CHAPTER ESSENCE AT A GLANCE:
Master user-defined composite records, memory address manipulation, pointer arithmetic, dynamic memory allocation with new and delete, and self-referential linked data structures.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Structures and Heterogeneous Aggregation:** A structure is a user-defined compound data type that bundles logically related variables of different primitive data types together under a single meaningful identifier.

• **Address and Dereference Mechanics:** Memory addresses identify exact storage locations; the address-of operator retrieves an address, whereas the dereference operator accesses or alters the data value held at that targeted memory location.

• **Pointer Member Navigation:** When referencing structure attributes through a pointer variable, the arrow selection operator provides a direct shorthand alternative to explicit pointer dereferencing followed by dot notation.

• **Dynamic Heap Lifecycle:** The dynamic allocation operator reserves fresh memory on the free store heap during program execution, while the deallocation operator releases that memory back to the operating system to prevent progressive memory exhaustion.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$*\text{ptr} \iff \text{Value stored at targeted memory address}$$

Pointer dereference operator: accesses data at address

$$\text{ptr} \rightarrow \text{member} \iff (*\text{ptr}).\,\text{member}$$

Structure pointer arrow selection operator equivalence

$$\&(\text{variable}) \iff \text{Memory address in hexadecimal}$$

Address-of operator: returns memory address of operand

$$\text{Allocated Elements} = \frac{\text{Total Dynamic Bytes}}{\text{Single Element Size}}$$

Dynamic heap memory calculation using sizeof operator

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Never omit the terminating semicolon after the closing curly brace of a struct definition; doing so triggers cascading compilation errors.

• **Alert:** Always reclaim dynamically allocated heap memory with delete or delete[] to avoid silent memory leaks in long-running processes.

• **Alert:** Avoid dangling pointers by explicitly assigning ptr = NULL or nullptr immediately after deallocation.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Define a structure in C++. Explain structure declaration and syntax with an example. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Definition of Structure):** A structure is a user-defined heterogeneous data type in C++ that groups related variables of different data types under a single name. Keyword: struct. **[1 Mark]**

• **Step 2 (Declaration & Access Syntax):** struct Student { int rollNo; char name[30]; float marks; }; Student s1; s1.rollNo = 101; strcpy(s1.name, 'Arun'); s1.marks = 95.5; Member elements are accessed using the dot (.) member access operator. **[1 Mark]**

• **Key Exam Result:** struct Student { int roll; char name[20]; float marks; }; Access via . operator

$$\text{ptr} \rightarrow \text{member} \iff (*\text{ptr}).\,\text{member}$$

Structure pointer arrow selection operator equivalence

2. How do you declare and initialize an array of structures? Give a C++ code snippet. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Array of Structures Concept):** Used to store homogeneous collections of structure variables (e.g. details of 50 students). **[1 Mark]**

• **Step 2 (Code Example):** struct Student { int roll; float marks; }; Student s[3] = { {101, 85.0}, {102, 92.5}, {103, 78.0} }; for(int i = 0; i < 3; i++) cout << s[i].roll << ' ' << s[i].marks << endl; **[2 Marks]**

• **Key Exam Result:** Student s[50]; accessed using loop: s[i].roll, s[i].marks

$$\text{ptr} \rightarrow \text{member} \iff (*\text{ptr}).\,\text{member}$$

Structure pointer arrow selection operator equivalence

3. Explain the concept of nested structures in C++ with suitable syntax. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Nested Structure Concept):** A structure embedded as a member inside another structure definition. **[1 Mark]**

• **Step 2 (Syntax Example):** struct Date { int day, month, year; }; struct Employee { int empId; char name[30]; Date dob; }; Employee e1; e1.empId = 501; e1.dob.day = 15; e1.dob.month = 8; e1.dob.year = 1995; **[1 Mark]**

• **Key Exam Result:** Outer.Inner.member access: e1.dob.day = 15;

$$\text{ptr} \rightarrow \text{member} \iff (*\text{ptr}).\,\text{member}$$

Structure pointer arrow selection operator equivalence

4. Explain memory allocation and byte alignment for structures in C++. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Memory Alignment Concept):** Total memory is the sum of sizes of individual member variables plus structure padding for hardware alignment. **[1 Mark]**

• **Step 2 (Size Calculation):** struct Item { int code; char cat; float price; }; sizeof(int) = 4 bytes, sizeof(char) = 1 byte, sizeof(float) = 4 bytes. Total = 4 + 1 + 3 (padding) + 4 = 12 bytes (or 9 bytes without padding). Evaluated using sizeof(Item). **[2 Marks]**

• **Key Exam Result:** Total memory evaluated via sizeof() operator with member alignment

$$\text{ptr} \rightarrow \text{member} \iff (*\text{ptr}).\,\text{member}$$

Structure pointer arrow selection operator equivalence

5. What is a pointer in C++? Explain the address-of (&) and dereference (*) operators. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Pointer Definition):** A pointer is a variable that stores the memory address of another variable. **[1 Mark]**

• **Step 2 (Operators):** Address-of operator (&): returns address of operand (&x). Dereference or indirection operator (*): accesses the value stored at the address pointed to (*p). int a = 10; int *p = &a; cout << *p; // prints 10. **[1.5 Marks]**

• **Step 3 (Syntax):** DataType *pointerVariable; (e.g. float *ptr;). **[0.5 Mark]**

• **Key Exam Result:** & (address-of) and * (dereference/value-at-address)

$$*\text{ptr} \Longleftrightarrow \text{Value stored at targeted memory address}$$

Pointer dereference operator: accesses data at address

6. Explain pointer arithmetic in C++. What operations are allowed and disallowed on pointers? [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Permitted Operations):** 1. Adding an integer to a pointer (p + 1 moves to next element: p + 1 × sizeof(type)). 2. Subtracting an integer from a pointer (p - 1). 3. Subtracting two pointers of same type (gives number of elements between them). 4. Pointer increment (p++) and decrement (p--). **[2 Marks]**

• **Step 2 (Illegal Operations):** Addition of two pointers, multiplication or division of pointers, adding float to pointer. **[1 Mark]**

• **Key Exam Result:** Valid: p+i, p-i, p1-p2, ++p; Invalid: p1+p2, p*i, p divided by i

$$*\text{ptr} \Longleftrightarrow \text{Value stored at targeted memory address}$$

Pointer dereference operator: accesses data at address

7. Explain the relationship between pointers and one-dimensional arrays in C++ with code. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Array-Pointer Equivalence):** The name of an array without brackets is a constant pointer pointing to the first element (arr == &arr[0]). **[1 Mark]**

• **Step 2 (Traversal Code):** int arr[5] = {10, 20, 30, 40, 50}; int *p = arr; for(int i = 0; i < 5; i++) { cout << *(p + i) << ' '; } Access via *(arr + i) is identical to arr[i]. **[2 Marks]**

• **Key Exam Result:** *(arr + i) is equivalent to arr[i]

$$*\text{ptr} \Longleftrightarrow \text{Value stored at targeted memory address}$$

Pointer dereference operator: accesses data at address